

Tech Nation Visa - Exceptional Promise

Criteria: Optional Criteria 3 (Significant Technical and Entrepreneurial Contribution)

Applicant: Odeyemi Olusegun Israel

Evidence 10: Cross-Chain Interoperability & SDK Distribution

1. Executive Summary

I built: compliance enforcement across seven simultaneous blockchain networks — closing the cross-chain evasion gap where a blacklisted wallet moves freely to a different chain — and two developer SDKs that compress third-party integration from weeks of custom infrastructure work to hours, exposing nine specialist AI compliance services behind a single import.

The problem: conventional compliance tools enforce rules on one blockchain. A blacklisted wallet simply moves assets to a different network and transacts freely. I solved this by propagating risk scores and block decisions across seven networks via LayerZero (a cross-blockchain messaging protocol that transmits signed data packets between networks) — a wallet blocked anywhere is blocked everywhere. EVM (Ethereum Virtual Machine) is the shared code standard across Ethereum, Polygon, Arbitrum, Optimism, Base, BSC, and Avalanche, meaning enforcement code written once applies to all of them.

Two developer SDKs — TypeScript (`@amttp/client-sdk`) and Python (`amttp`) — accompany the core system, exposing the full compliance stack behind a simple import. Neither wraps an existing library; both were written from scratch.

Cross-Chain Contract Integration (from `contracts/AMTTPCrossChain.sol`):

```
contract AMTTPCrossChain is Initializable, OwnableUpgradeable,
UUPSUpgradeable, ILayerZeroReceiver {
    mapping(address => mapping(uint16 => uint256))
    public crossChainRiskScores;
    mapping(address => bool) public globallyBlocked;
    ILayerZeroEndpoint public lzEndpoint;

    function sendRiskScore(
        uint16 _dstChainId, address _targetAddress,
        uint256 _riskScore, bytes calldata _adapterParams
    ) external payable whenNotPaused nonReentrant {
        require(_riskScore <= 1000, "Invalid risk score");
        bytes memory payload = abi.encode(
            MSG_RISK_SCORE, _targetAddress, _riskScore);
        lzEndpoint.send{value: msg.value}(
            _dstChainId, trustedRemotes[_dstChainId],
            payload, payable(msg.sender), address(0),
            _adapterParams);
    }
}
```

Figure 1: `AMTTPCrossChain.sol` — upgradeable Solidity contract implementing LayerZero cross-chain message passing. A risk score or block decision on one of 7 EVM networks is encoded, signed, and broadcast to all connected chains. `nonReentrant` and `whenNotPaused` guards prevent replay and denial-of-service.

2. Developer SDK Distribution

Two developer integration kits were designed and released alongside the core system:

- **TypeScript SDK** (@amttp/client-sdk): For web and application developers. Provides AI-powered risk scoring, blockchain enforcement integration, identity compliance checks, and support for multiple blockchain networks, accessible in a few lines of code.
- **Python SDK** (amttp): For data scientists and backend engineers. Provides the same capabilities using the Python programming language, which is standard in financial analytics and data science teams.

TypeScript SDK Example (from packages/client-sdk/README.md):

```
import { MLService, PolicyAction, utils } from '@amttp/client-sdk';
const mlService = new MLService('http://localhost:8000');
const risk = await mlService.scoreTransaction('tx.001', {
  amount: 1.5,           // ETH
  velocity_24h: 3,
  account_age_days: 90,
});
console.log('Risk:  ${utils.formatRiskScore(risk.riskScore)}'); // "97.2%"
console.log('Action:  ${PolicyAction[risk.action]}');           // "ESCROW"
```

Python SDK Example (from packages/python-sdk/README.md):

```
from amttp import AMTTPClient, AMTTPConfig
config = AMTTPConfig(
  rpc_url="https://mainnet.infura.io/v3/YOUR_KEY",
  oracle_url="https://oracle.amttp.io",
  ml_api_url="https://ml.amttp.io",
)
client = AMTTPClient(config)
risk = client.score_transaction(
  to="0x742d35Cc...", value=1_000_000_000_000_000,
  features={"velocity_24h": 5, "account_age_days": 30}
)
print(f"Risk Score:  {risk.risk_score:.2%}")      # "Risk Score:  94.12%"
print(f"Action:  {risk.action.name}")              # "Action:  ESCROW"
```

Figure 2: TypeScript and Python SDK quick-start examples. A third-party developer can score a transaction against AMTTP's full compliance stack — ML risk, FCA sanctions, policy engine, and cross-chain data — in under 10 lines of code in their preferred language. The Python SDK targets financial analytics teams; the TypeScript SDK targets web and application developers.

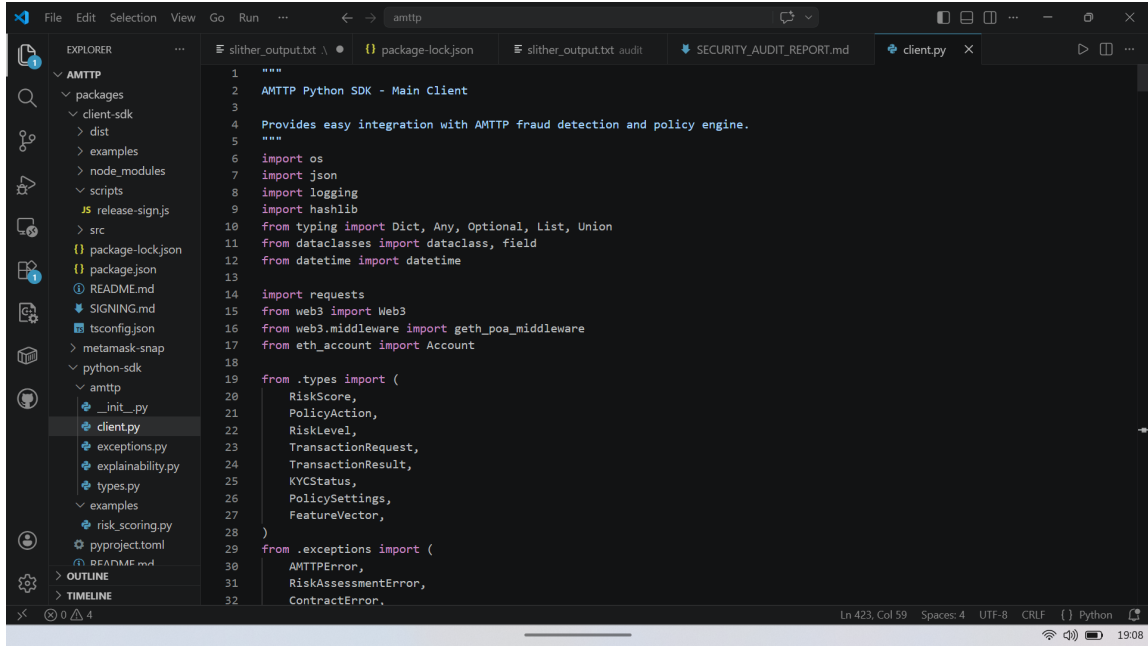


Figure 3: VS Code workspace showing both SDKs fully structured and ready for distribution. Both packages are sole-authored: no collaborative commits, no fork from an existing open-source SDK. Each wraps the full nine-service AMTTP compliance stack behind a single import.

3. Commercial Readiness and UK Ecosystem Value

Cross-chain enforcement and distribution-ready SDKs transform AMTTP from a research system into a deployable compliance product. The architectural decision to implement enforcement at the cross-chain messaging layer — rather than as a post-settlement reporting tool — means a block decision propagates to all seven networks within the same transaction cycle. A wallet cannot evade by moving chains.

The two SDKs directly address the principal integration barrier for regulated institutions: a bank, payment processor, or fintech operating on any of the seven supported networks can integrate the full nine-service compliance stack within hours, in their preferred language, without rebuilding or understanding the underlying infrastructure. This eliminates a cost and complexity barrier that would otherwise require a dedicated blockchain engineering team. For UK digital-finance institutions operating under FCA digital-asset rules, Travel Rule obligations, or MiCA cross-border requirements, the enforcement layer and its distribution tooling represent production-ready infrastructure they can adopt immediately. I designed, built, and sole-authored every component.